

client/client_core.py

```
# client_core.py
# í .° ... ì ì ,í , TCP í μì °°°

import json
import socket
import threading
import os
import sys
sys.path.append(os.path.dirname(os.path.dirname(os.path.abspath(__file__))))
from env_utils import DEFAULT_HOST, DEFAULT_PORT, BUFSIZE

class TCPClient:
    """TCP/IP ì ì... í .° ... ì ì ,í , (ì ì†>° „ ì ì °£†° °°'ì )"""

    def __init__(self):
        self.lock = threading.Lock()
        self.is_connected = False
        self.host = None
        self.port = None

    def connect(self, host, port):
        """ì °† ì °£†° í ì /í , í í ,ì /í ,/í <í ,°§ £,°ì μ"""
        try:
            # í °† °§ í ì /í , ì °£†°
            with socket.create_connection((host, port), timeout=5):
                pass
            with self.lock:
                self.host = host
                self.port = port
                self.is_connected = True
            return True
        except Exception as e:
            print(f"ì °£†° ì /í ¢: {e}")
            return False

    def disconnect(self):
        """ì °† ì °£†° í ì (° ...°f<ì ì °£†°°§ í ì )"""
        with self.lock:
            self.host = None
            self.port = None
            self.is_connected = False

    def _ensure_target(self):
        if not self.is_connected or not self.host or not self.port:
            raise RuntimeError("ì °† ì ì °£†°° ì . ì ì§ ì ì μ° ° /.")

    def send_command(self, command, data=None):
        """ì °† ì °° ° „ ì ì ì (ì ì†>° „ ì ì ì... ì <ì ')"""
        try:
            self._ensure_target()
        except RuntimeError as e:
            return {"status": "error", "message": str(e)}
```

Client/client_core.py

```
payload = {
    "command": command,
    "data": data or {}
}
body = json.dumps(payload, ensure_ascii=False).encode("utf-8")
header = len(body).to_bytes(4, byteorder="big")

with self.lock:
    # STT ì† °f<° OpenAI API í ,ì¶ °i ì ¤° ì · ¤-,°f· ì ì ì ...°fl °i í ì ì ì
    with socket.create_connection((self.host, self.port), timeout=300) as sock:
        # ì ì i
        sock.sendall(header)
        sock.sendall(body)

        # ì ° μ ì ì (STT ì† °f< ì ¤° ¤‡ ° /í ì < í ì ì ì ì f ¤° )
        sock.settimeout(300.0) # 5°¶ í ì ì ì
        length_bytes = sock.recv(4)
        if len(length_bytes) != 4:
            return {"status": "error", "message": "ì ° μ ¤,,ì · ì ì ì /í ¤"}

        resp_len = int.from_bytes(length_bytes, byteorder="big")
        resp_data = b""
        while len(resp_data) < resp_len:
            chunk = sock.recv(min(BUFSIZE, resp_len - len(resp_data)))
            if not chunk:
                break
            resp_data += chunk

        if len(resp_data) < resp_len:
            return {"status": "error", "message": "ì ° μ ° ì · í ° °¶ ì ì-"}

        return json.loads(resp_data.decode("utf-8"))

except Exception as e:
    return {"status": "error", "message": str(e)}

def send_file(self, file_path):
    """í ì ... ì ì i (ì ì†>° „ ì ì ì ... ì < i ')"""
    try:
        self._ensure_target()
    except RuntimeError as e:
        return {"status": "error", "message": str(e)}

    try:
        if not os.path.exists(file_path):
            return {"status": "error", "message": "í ì ... ì ì°¾ ì ì ì μ° ° /."}

        file_size = os.path.getsize(file_path)
        filename = os.path.basename(file_path)

        with self.lock:
            # í ì ... ì ì i ì í ° í ì ...° ¤‡ ° /í ì < í ì ì ì ì f ¤°
            with socket.create_connection((self.host, self.port), timeout=300) as sock:
                sock.settimeout(300.0) # 5°¶ í ì ì ì
```

```

Client/client_core.py
    filename_bytes = filename.encode('utf-8')
    sock.sendall(filename_bytes + b'\0')

    # Wait for ready signal
    ready_signal = sock.recv(5)
    if ready_signal != b"READY":
        return {"status": "error", "message": "Invalid ready signal"}

    # Send file size
    file_size_str = str(file_size).encode('utf-8')
    sock.sendall(file_size_str + b'\0')

    # Send file content
    sent_size = 0
    with open(file_path, 'rb') as f:
        while sent_size < file_size:
            chunk = f.read(BUFSIZE)
            if not chunk:
                break
            sock.sendall(chunk)
            sent_size += len(chunk)

    # Receive response
    response = sock.recv(7)
    if response == b"SUCCESS":
        return {"status": "success", "message": "File uploaded successfully"}
    else:
        return {"status": "error", "message": "File upload failed"}

except Exception as e:
    return {"status": "error", "message": str(e)}

```